

RUTA DE APRENDIZAJE · PROGRAMACIÓN EN JAVA

Java 2026 — De cero a experto

CAPÍTULO

01

Introducción a Java y fundamentos del lenguaje

De cero a tu primer programa: historia, herramientas, sintaxis y tus primeras líneas de código.

EN ESTE CAPÍTULO APRENDERÁS

Historia de Java · Ediciones SE / EE / ME · JVM, JRE y JDK · Instalación del entorno ·
“Hola, Mundo” · Compilación y ejecución · Sintaxis, variables y tipos · Entrada con Scanner

4 h

TIEMPO ESTIMADO

8

SECCIONES

Inicial

NIVEL

4

RETOS

Libro-tutorial paso a paso · Edición a color 2026

Cómo usar este capítulo

Este no es un texto para leer de corrido: es un taller para hacer. A lo largo del capítulo encontrarás cajas de colores que cumplen funciones específicas. Apréndete su código de colores y sabrás de un vistazo qué se espera de ti en cada momento.

	Caja	Para qué sirve
	Objetivos	Lo que sabrás hacer al terminar. Léelos antes de empezar.
	¿Por qué importa?	El sentido práctico del tema: dónde y cuándo lo usarás de verdad.
	Concepto clave	La idea central que debes retener sí o sí.
	Manos a la obra	Ejercicio guiado que debes ejecutar en tu computador.
	Tip pro	Atajos y buenas prácticas que usan los profesionales.
	Cuidado	Errores frecuentes de principiantes y cómo evitarlos.
	Punto de control	Verifica que todo funciona antes de avanzar.
	Glosario exprés	Definiciones rápidas de los términos nuevos.
	Para reflexionar	Preguntas para pensar y afianzar lo aprendido.

Regla de oro del taller

No copies y pegues el código: escríbelo a mano. El teclado enseña lo que los ojos no ven, y los errores que cometerás al teclear son exactamente los que aprenderás a resolver.

Mapa del capítulo

Ocho paradas, en orden. Cada una se apoya en la anterior: al final del recorrido habrás escrito, compilado y ejecutado programas interactivos.

Nº	Sección	Al terminarla podrás...
1	¿Qué es Java? Una breve historia	Explicar qué es Java, de dónde viene y por qué sigue vigente.
2	Las ediciones: SE, EE y ME	Elegir la edición correcta para cada tipo de proyecto.
3	El trío mágico: JVM, JRE y JDK	Distinguir las tres siglas sin volver a confundirlas.
4	Instala tu entorno de trabajo	Tener un JDK funcionando y un editor listo para programar.
5	Tu primer programa: “Hola, Mundo”	Escribir y entender línea a línea tu primer programa.

Nº	Sección	Al terminarla podrás...
6	Compilación y ejecución	Usar javac y java, y explicar qué es el bytecode.
7	Sintaxis básica del lenguaje	Declarar variables con el tipo correcto y nombrarlas bien.
8	Entrada y salida con Scanner	Crear programas interactivos que lean datos del usuario.

En este capítulo aprenderás a...

- Entender qué es Java, de dónde viene y por qué sigue siendo tan popular.
- Distinguir entre JVM, JRE y JDK (sin volver a confundirlos nunca).
- Instalar el entorno, escribir, compilar y ejecutar tu primer programa.
- Dominar la sintaxis básica: identificadores, palabras clave, comentarios, variables, tipos y literales.
- Leer datos del usuario con la clase Scanner.

1

¿Qué es Java? Una breve historia

El lenguaje que prometió escribirse una vez y ejecutarse en todas partes

💡 ¿Por qué importa?

Antes de escribir una sola línea conviene saber qué terreno pisas: Java mueve la banca, el comercio electrónico, miles de servidores y millones de teléfonos Android. Entender su origen te explica sus decisiones de diseño... y por qué te pedirá compilar antes de ejecutar.

Java es un lenguaje de programación de propósito general, orientado a objetos y diseñado para que el mismo programa pueda ejecutarse en cualquier dispositivo sin reescribirlo. Esa promesa se resume en su lema más famoso:

“Write once, run anywhere” (*escribe una vez, ejecuta en cualquier lugar*)

Nació a principios de los años 90 en Sun Microsystems, de la mano de James Gosling y su equipo. El proyecto se llamaba originalmente “Oak” (roble), pero terminó adoptando el nombre Java, inspirado en el café que tanto consumían sus creadores; por eso su logo es una taza humeante.



Figura 1.1 — Una línea de tiempo simplificada de la historia de Java.

☕ Dato curioso

El icono de la taza de café no es casualidad. “Java” es también el nombre de una isla de Indonesia famosa por su café, y de ahí viene tanto el nombre del lenguaje como su logotipo.

¿Por qué sigue siendo tan popular?

Fortaleza	Qué significa en la práctica
🌐 Multiplataforma	El mismo archivo compilado corre en Windows, macOS, Linux o un servidor en la nube.
🛡️ Robusto y seguro	Gestiona la memoria por ti (recolector de basura) y evita muchos errores comunes.
🧩 Ecosistema gigante	Millones de bibliotecas, frameworks (Spring, Android) y una comunidad enorme.

Fortaleza	Qué significa en la práctica
 Presencia real	Es la base de Android, de banca, comercio electrónico y sistemas empresariales de medio mundo.

Para reflexionar

Piensa en tres aplicaciones que uses a diario. ¿Cuáles crees que podrían estar construidas sobre Java? Al final del capítulo, investiga una y comprueba tu hipótesis.

2

Las ediciones de Java: SE, EE y ME

Un mismo lenguaje, tres cajas de herramientas

Java no es un único producto monolítico. Se organiza en “ediciones”, cada una pensada para un tipo de proyecto. Todas comparten el mismo lenguaje base; lo que cambia es el conjunto de bibliotecas y herramientas que incluyen.

Edición	Para qué sirve	Ejemplos típicos
Java SE	Standard Edition. El núcleo del lenguaje y las bibliotecas fundamentales. Es donde empiezan todos.	Aplicaciones de escritorio y de consola, fundamentos.
Java EE	Enterprise Edition (hoy Jakarta EE). Amplía SE con tecnologías para servidores y grandes sistemas.	Aplicaciones web, servicios empresariales, APIs.
Java ME	Micro Edition. Una versión reducida para dispositivos con recursos limitados.	Dispositivos embebidos, IoT, sistemas pequeños.

Para empezar, usa Java SE

En este libro trabajaremos siempre con Java SE. Es el cimiento de todo lo demás: cuando domines SE, saltar a EE o a Android te resultará natural.

3

El trío mágico: JVM, JRE y JDK

Tres siglas, una muñeca rusa

¿Por qué importa?

Estas tres siglas aparecen en toda descarga, tutorial y entrevista de trabajo. Confundirlas es el error número uno de los principiantes; dominarlas te ahorrará horas de instalación fallida.

La forma más fácil de recordarlas es pensar en muñecas rusas: una contiene a la otra.

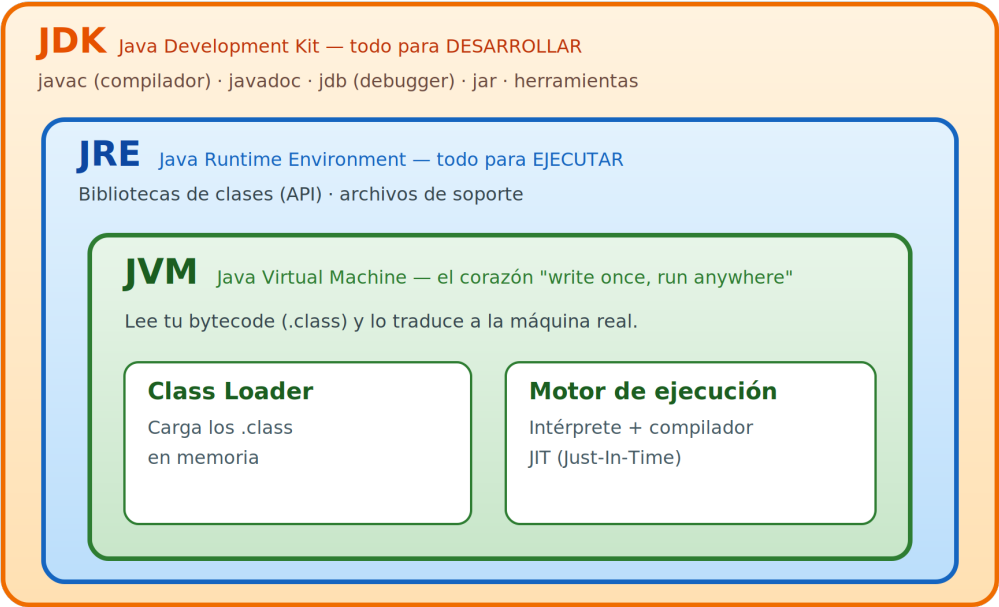


Figura 1.2 — El JDK contiene al JRE, y el JRE contiene a la JVM.

Sigla	Nombre completo	Qué es y para qué sirve
JVM	Java Virtual Machine	Una “máquina” imaginaria (un programa) que sabe ejecutar el código compilado de Java, llamado bytecode. Hace posible la portabilidad: hay una JVM distinta por sistema operativo, pero todas entienden el mismo bytecode.
JRE	Java Runtime Environment	La JVM más las bibliotecas estándar y archivos de apoyo necesarios para ejecutar programas. Si solo quieres usar aplicaciones (no crearlas), el JRE basta.
JDK	Java Development Kit	El paquete completo para desarrollar: todo el JRE más el compilador javac, el depurador, javadoc y más. Como vas a programar, esto es lo que instalarás.

🔑 **Regla para no olvidarlo nunca**

¿Solo vas a **EJECUTAR** programas? → necesitas el **JRE**.

¿Vas a **CREAR** programas? → necesitas el **JDK** (que ya trae el JRE dentro).

📖 **Glosario exprés**

Bytecode: formato intermedio (.class) al que se compila tu código y que la JVM sabe ejecutar.

Compilador: programa que traduce tu código fuente a un formato ejecutable; en Java, javac.

Portabilidad: capacidad de un mismo programa de correr en distintos sistemas sin cambios.

4

Instala tu entorno de trabajo

JDK + editor: todo lo que necesitas para programar

Para escribir tu primer programa necesitas dos cosas: el JDK (para compilar y ejecutar) y un editor de código donde escribir. Sigue estos pasos.

Paso 1 — Descarga e instala un JDK

- Descarga una versión LTS (de soporte extendido) como Java 17 o Java 21, desde Oracle o desde una distribución abierta como Adoptium / Eclipse Temurin.
- Ejecuta el instalador para tu sistema operativo (Windows, macOS o Linux) y sigue el asistente.

Paso 2 — Verifica la instalación

Abre una terminal (Símbolo del sistema en Windows, Terminal en macOS/Linux) y escribe estos comandos. Si el JDK quedó bien instalado, verás la versión:

```
Terminal

1 java -version
2 javac -version

Conola / Terminal
java version "21.0.2" 2024-01-16 LTS
javac 21.0.2
```

⚠ ¿El comando no funciona?

Si la terminal responde “comando no encontrado”, casi siempre es por la variable de entorno PATH. Asegúrate de que la carpeta bin del JDK esté incluida en el PATH del sistema y vuelve a abrir la terminal.

Paso 3 — Elige un editor o IDE

Momento	Herramienta recomendada
Para empezar	VS Code con la extensión de Java, o incluso un editor de texto simple.
Cuando crezcas	Un IDE completo como IntelliJ IDEA o Eclipse, que autocompletan código y detectan errores al instante.

 Punto de control

Antes de seguir, verifica que puedes marcar las tres casillas:

- ☐ java -version muestra una versión 17 o superior.
- ☐ javac -version responde sin error.
- ☐ Tienes un editor instalado y sabes crear un archivo .java con él.

5

Tu primer programa: “Hola, Mundo”

La tradición que inaugura a todo programador

Es tradición que el primer programa en cualquier lenguaje solo muestre un saludo en pantalla. Crea un archivo llamado HolaMundo.java y escribe exactamente esto:

```
● ● ●  HolaMundo.java

1  public class HolaMundo {
2      public static void main(String[] args) {
3          // Esta línea imprime un mensaje en pantalla
4          System.out.println("¡Hola, Mundo!");
5      }
6  }
```

Anatomía de tu primer programa

Cada palabra de ese programa tiene un porqué. Vamos a diseccionarlo pieza por pieza:

Fragmento	Qué hace
public class HolaMundo	Declara una clase pública llamada HolaMundo. En Java, todo el código vive dentro de una clase, y el archivo debe llamarse igual que la clase: HolaMundo.java.
public static void main(...)	El método main es el punto de entrada: la JVM siempre empieza a ejecutar aquí. Debe escribirse exactamente así.
String[] args	Un arreglo de textos donde llegan los argumentos que se pasan al programa desde la terminal.
System.out.println(...)	Imprime en pantalla el texto que le des y salta a la línea siguiente. Usa print(...) si no quieres el salto.
"¡Hola, Mundo!"	Una cadena de texto (String). Todo texto literal va entre comillas dobles.
{ } ;	Las llaves delimitan bloques de código; el punto y coma cierra cada instrucción.

🔑 La llave de oro

El nombre del archivo debe coincidir EXACTAMENTE con el nombre de la clase pública, respetando mayúsculas y minúsculas: `HolaMundo.java` ↔ `class HolaMundo`. Si no coinciden, el compilador se quejará.

🔧 Manos a la obra

Escribe `HolaMundo.java` en tu editor (¡a mano, sin copiar!). Luego cambia el mensaje por un saludo con tu nombre, guarda y vuelve a ejecutarlo en la sección siguiente. Romperlo a propósito también enseña: quita el punto y coma y lee el error del compilador.

6

Compilación y ejecución

Del código fuente al resultado en dos comandos

Java no ejecuta tu archivo `.java` directamente. Primero lo compila a un formato intermedio llamado bytecode (`.class`), y luego la JVM lo ejecuta. Son dos comandos distintos:

De tu teclado a la pantalla

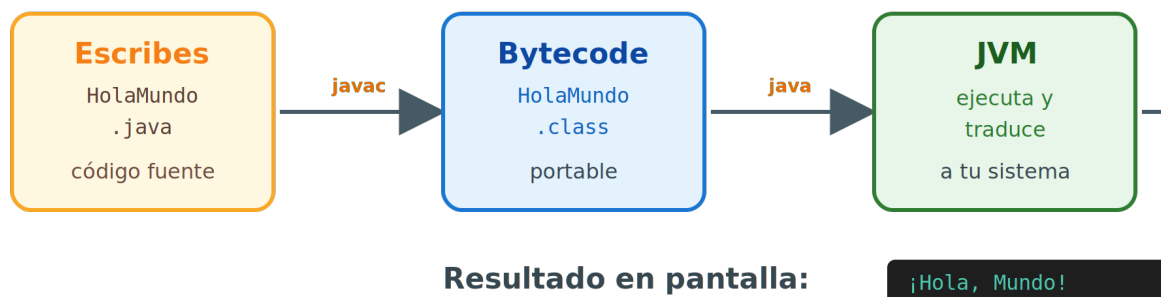


Figura 1.3 — Del código fuente al resultado: `javac` compila, `java` ejecuta.

Paso 1 — Compilar con `javac`

El compilador `javac` toma tu código fuente y produce un archivo `HolaMundo.class` con el bytecode:

```

Terminal
1 javac HolaMundo.java
  
```

Paso 2 — Ejecutar con `java`

Ahora lanzas la JVM con el comando `java`, pasándole el nombre de la clase (¡sin la extensión `.class`!):

```

Terminal

1 java HolaMundo

Consola / Terminal
¡Hola, Mundo!
    
```

⚡ Tip pro: ejecución directa desde Java 11

Desde Java 11 puedes ejecutar programas de un solo archivo sin compilarlos manualmente, escribiendo directamente: `java HolaMundo.java`. Internamente Java compila y ejecuta en un paso. ¡Perfecto para practicar!

✅ Punto de control

- ☐ Al compilar apareció un archivo `HolaMundo.class` en la carpeta.
- ☐ `java HolaMundo` imprime tu saludo en la terminal.
- ☐ Puedes explicar con tus palabras qué hace `javac` y qué hace `java`.

7

Sintaxis básica del lenguaje

Las reglas que se repiten en todo código Java

Ya viste un programa completo. Ahora desglosemos las reglas fundamentales que se repiten en todo código Java.

7.1 Identificadores

Un identificador es el nombre que le pones a tus clases, variables y métodos. Tienes libertad, pero con reglas:

- Pueden contener letras, dígitos, el guion bajo (`_`) y el signo de dólar (`$`).
- No pueden empezar por un dígito, ni contener espacios, ni ser una palabra reservada.
- Distinguen mayúsculas de minúsculas: `edad`, `Edad` y `EDAD` son tres nombres distintos.

✅ Válidos	❌ Inválidos
<code>nombreUsuario</code>	<code>2nombre</code> (empieza por dígito)
<code>edad_2</code>	<code>mi variable</code> (tiene espacio)
<code>\$precio</code>	<code>class</code> (palabra reservada)

⚡ Convención: camelCase y PascalCase

Por costumbre, las variables y métodos usan camelCase (miNotaFinal) y las clases usan PascalCase (CuentaBancaria). No es obligatorio para que compile, pero todo el mundo lo hace.

7.2 Palabras reservadas (keywords)

Son palabras que Java se reserva para sí mismo porque tienen un significado especial. No puedes usarlas como nombres de tus variables. Estas son algunas de las más usadas:

class	public	private	static
void	int	double	boolean
if	else	for	while
return	new	true	false
null	this	import	final

⚠ Ojo

Palabras como true, false y null técnicamente son literales reservados, no keywords, pero igual no puedes usarlas como nombres. El efecto práctico es el mismo: están prohibidas.

7.3 Comentarios

Los comentarios son notas para los humanos; la JVM los ignora por completo. Hay tres tipos:

```

Comentarios.java
1 // Comentario de una sola línea
2
3 /* Comentario
4    de varias líneas */
5
6 /**
7  * Comentario de documentación (Javadoc).
8  * Sirve para generar documentación automática.
9  */
    
```

7.4 Variables

Una variable es un espacio con nombre donde guardas un valor que puede cambiar. Para crearla, indicas su tipo, su nombre y, opcionalmente, un valor inicial:

```

Variables.java

1  int edad;           // declaración (sin valor aún)
2  edad = 25;         // asignación
3
4  int altura = 175;   // declaración + asignación juntas
5  double precio = 19.99;
6  boolean activo = true;
7  String nombre = "Ana";
    
```

Una variable es como una caja etiquetada

Imagina una caja con una etiqueta (el nombre) que solo admite cierto contenido (el tipo). Puedes cambiar lo que hay dentro, pero no meter algo que no corresponda: una caja de int no acepta texto.

7.5 Tipos de datos primitivos

Java tiene 8 tipos primitivos: los ladrillos más básicos para almacenar datos. No son objetos; guardan directamente el valor.

Tipo	Guarda	Tamaño	Ejemplo
byte	Entero muy pequeño	8 bits	-128 a 127
short	Entero pequeño	16 bits	hasta 32 767
int	Entero (el más usado)	32 bits	42
long	Entero grande	64 bits	9000000000L
float	Decimal de precisión simple	32 bits	3.14f
double	Decimal de doble precisión	64 bits	3.14159
char	Un solo carácter	16 bits	'A'
boolean	Verdadero o falso	1 bit	true / false

¿Y el texto?

String (para cadenas de texto) NO es un tipo primitivo: es una clase. Por eso se escribe con mayúscula. Lo usarás todo el tiempo, pero técnicamente pertenece a otra categoría.

7.6 Literales

Un literal es un valor escrito tal cual en el código. Cada tipo tiene su forma de escribirse:

```
● ● ● Literales.java

1 int decimal = 100;           // literal entero
2 int hexadecimal = 0xFF;      // base 16
3 int binario = 0b1010;        // base 2
4 long grande = 1000000L;      // La L lo marca como long
5 double pi = 3.14159;         // literal decimal
6 float tasa = 2.5f;           // La f lo marca como float
7 char letra = 'J';            // un carácter, comilla simple
8 boolean ok = true;           // literal booleano
9 String saludo = "Hola";      // literal de texto
```

⚠ Detalle fino

Un texto va entre comillas dobles ("Hola") y un solo carácter entre comillas simples ('J'). Confundirlas es uno de los errores más comunes al empezar.

✅ Punto de control

- ☐ Sé nombrar variables válidas y sigo camelCase.
- ☐ Reconozco los 8 tipos primitivos y sé cuándo usar int, double y boolean.
- ☐ Distingo 'J' (char) de "J" (String) sin dudar.

8

Entrada y salida básica con Scanner

Convierte tus programas en una conversación

💡 ¿Por qué importa?

Hasta ahora solo hemos mostrado información en pantalla (salida). Un programa que no escucha al usuario es un monólogo. Con Scanner tus programas empiezan a dialogar: piden datos, los procesan y responden.

La herramienta estándar para leer lo que el usuario escribe es la clase Scanner.

Paso 1 — Importar Scanner

Scanner vive en el paquete java.util, así que primero hay que importarlo al inicio del archivo:

```
● ● ● Entrada.java

1 import java.util.Scanner;
```

Paso 2 – Crear el Scanner y leer datos

Creamos un objeto Scanner conectado al teclado (System.in) y usamos sus métodos para leer distintos tipos:

```

Entrada.java

1  import java.util.Scanner;
2
3  public class Entrada {
4      public static void main(String[] args) {
5          Scanner sc = new Scanner(System.in);
6
7          System.out.print("¿Cómo te llamas? ");
8          String nombre = sc.nextLine();
9
10         System.out.print("¿Cuántos años tienes? ");
11         int edad = sc.nextInt();
12
13         System.out.println("Hola " + nombre + ", tienes " + edad);
14         sc.close();
15     }
16 }

```

Consola / Terminal

```

¿Cómo te llamas? Ana
¿Cuántos años tienes? 25
Hola Ana, tienes 25

```

Métodos más usados de Scanner

Método	Lee...
<code>nextLine()</code>	Una línea completa de texto (incluye espacios).
<code>next()</code>	Una sola palabra (hasta el primer espacio).
<code>nextInt()</code>	Un número entero.
<code>nextDouble()</code>	Un número decimal.
<code>nextBoolean()</code>	Un valor true o false.

⚠ La trampa clásica de `nextInt()` + `nextLine()`

Cuando lees un número con `nextInt()` y luego un texto con `nextLine()`, el segundo suele “saltarse” porque queda un salto de línea pendiente. La solución habitual es añadir un `sc.nextLine()` extra para consumirlo, o leer todo como texto y convertirlo. ¡No te asustes cuando te pase!

 Manos a la obra

Modifica Entrada.java para que además pregunte la ciudad donde vive el usuario (después de la edad) y la incluya en el saludo final. Vivirás en carne propia la trampa de `nextInt()` + `nextLine()`... y la resolverás.

 RETOS DEL CAPÍTULO

Pon a prueba lo aprendido. Intenta resolver cada reto antes de mirar la pista: escribir código es la única forma de aprenderlo.

 Reto 1 · Tarjeta de presentación ★★★

Escribe un programa que imprima en tres líneas tu nombre, tu lenguaje favorito y un mensaje motivador.

 **Pista:** Usa tres `System.out.println(...)` seguidos.

 Reto 2 · Calculadora de edad ★★★

Pide al usuario el año en que nació y el año actual con `Scanner`, calcula su edad y muéstrala con un mensaje.

 **Pista:** $Edad = añoActual - añoNacimiento$. Usa `nextInt()` para leer cada año.

 Reto 3 · Detective de tipos ★★★

Declara una variable de cada tipo primitivo con un valor de ejemplo e imprímelas todas. ¿Cuál no compila si le das el valor equivocado?

 **Pista:** Prueba a meter un decimal en un `int` y observa el error del compilador.

 Reto 4 · Conversor interactivo ★★★

Pide al usuario una temperatura en grados Celsius (decimal) y conviértela a Fahrenheit con la fórmula $F = C \times 9/5 + 32$.

 **Pista:** Lee con `nextDouble()`. Cuidado con dividir enteros: usa `9.0/5.0`.

✓ RESUMEN DEL CAPÍTULO

Lo esencial que te llevas

- Java es multiplataforma gracias al bytecode y la JVM: “write once, run anywhere”.
- Java se organiza en ediciones: SE (núcleo), EE (empresarial) y ME (dispositivos). Empezamos por SE.
- JDK contiene al JRE, y el JRE contiene a la JVM. Para programar, instala el JDK.
- Se programa en dos pasos: javac compila el .java a bytecode (.class) y java lo ejecuta en la JVM.
- Todo programa arranca en public static void main(String[] args), dentro de una clase que se llama igual que el archivo.
- Las variables tienen tipo y nombre; Java ofrece 8 tipos primitivos y la clase String para texto.
- Los identificadores siguen reglas (no empezar por dígito, sin espacios) y convenciones (camelCase, PascalCase).
- Scanner (java.util) permite leer datos del usuario: nextLine(), nextInt(), nextDouble()...

Tu caja de comandos

Comando	Qué hace
<code>java -version</code>	Muestra la versión de Java instalada (verifica el JRE/JVM).
<code>javac -version</code>	Muestra la versión del compilador (verifica el JDK).
<code>javac MiClase.java</code>	Compila el código fuente y genera MiClase.class (bytecode).
<code>java MiClase</code>	Ejecuta la clase compilada en la JVM (sin extensión).
<code>java MiClase.java</code>	Desde Java 11: compila y ejecuta un archivo único en un solo paso.

Autoevaluación: ¿puedo decir que sí?

Marca cada afirmación solo si puedes demostrarla escribiendo código o explicándosela a alguien más. Si alguna queda sin marcar, vuelve a la sección indicada antes de pasar al Capítulo 2.

Afirmación	Repasa en...
☐ Puedo explicar qué son la JVM, el JRE y el JDK, y cuál instalé.	Sección 3
☐ Escribí, compilé y ejecuté HolaMundo.java sin ayuda.	Secciones 5–6

Afirmación	Repasa en...
☐ Sé declarar variables de los tipos int, double, boolean, char y String.	Sección 7
☐ Distingo un identificador válido de uno inválido a simple vista.	Sección 7.1
☐ Puedo leer un texto y un número del usuario con Scanner.	Sección 8
☐ Resolví al menos 3 de los 4 retos del capítulo.	Retos